Tutorial – 01

1. Step 1: Understand and Define the Problem (Analyze)

Design a logic-based system to manage a smart parking barrier on a university entrance. The system should be able to do the following functions,

- Automobile barrier opening upon staff and students with registration.
- Select the number of cars present in the parking lot.
- Show the amount of parking spaces available.
- Sound an alarm when an attempt is made to get into a vehicle when the parking lot is full.

The system must check the vehicles coming in, whether the user is registered and whether parking spaces are under occupation before opening the barrier.

2. Step 2: Organize and Describe the Data

This system has 4 inputs**.**

- REG – Vehicle belongs to a registered staff/student (1 = Registered)
- ENTRY – Vehicle detected at the entrance (1 = Entrance Detected)
- EXIT – Vehicle leaving the parking lot (1 = Exit Detected)
- FULL – Parking lot is full (1 = Parking is full)

The system has 3 outputs**.**

- BARRIER – Barrier control (1 = Open, 0 = Closed)
- COUNT – Car counter (increase/decrease depending on entry or exit)
- ALERT – Alert signal (1 = Alert ON)

3. Step 3: Plan the Solution (Design the Algorithm)

- Monitor the vehicles at the entrance.
- Check if the vehicle is registered. (REG = 1)
- Check if the parking lot is full. (FULL = 0)
- If a vehicle arrives, and is registered, and parking is not full – BARRIER = 1.
- Increase the vehicle count when a new vehicle enters.
- If a vehicle exits - decrease the car count.
- If a vehicle tries to enter when FULL = 1: - trigger an alert and keep the barrier closed.

4. Step 4: Implement the Solution (Word Coding)

The logic system can be implemented using:

- Vehicle detection sensors
- RFID card reader for registered users
- Microcontroller (such as Arduino)
- Barrier motor control
- Parking counter system
- LED/LCD display for available spaces
- Alarm or buzzer for alerts

The microcontroller processes sensor inputs and executes the programmed logic to control the barrier and alerts.

5. Step 5: Test and Refine the Solution (Debug and Verify)

1. **Registered vehicle entering when parking is available**
    - Barrier opens
    - Car count increases

2. **Unregistered vehicle attempting entry**
    - Barrier remains closed
3. **Vehicle attempting entry when parking is full**
    - Barrier remains closed
    - Alert is triggered
4. **Vehicle exiting the parking lot**
    - Car count decreases
    - Available parking spaces update

AI Agent Integration

1. Refine your logic or Word Code

- Ensuring the barrier only opens when the vehicle is registered and parking is not full.
- Adding a clear condition for exit detection to reduce the car count.
- Updating the parking display whenever the count changes.

2. Generate alternative solutions

- Using RFID cards to identify registered vehicles.
- Installing number plate recognition cameras instead of manual registration checks.
- Using two sensors at the gate (entry and exit sensors) to improve vehicle counting accuracy.

3. Explore real-world implementation

- Arduino or Raspberry Pi as the main controller
- RFID reader to verify registered staff and students

- Ultrasonic or infrared sensors to detect vehicle entry and exit
- Servo motor to control the parking barrier
- LCD or LED display to show the number of available parking spaces
- Buzzer or alarm to alert when parking is full

4. Reflect on ethics and limitations

- AI suggestions may not always be fully accurate, so human verification is necessary.
- Automated systems that collect vehicle data may raise privacy concerns.
- Hardware failures or incorrect sensor readings could cause incorrect parking counts.

Therefore, AI should be used as a support tool, while the final design and verification should be done by humans.